

Apêndice B — Configuração inicial de projeto com Reflex usando venv

Uso: execute este apêndice quando o grupo tiver escolhido **Reflex** como tecnologia obrigatória do frontend. Esta versão usa o ambiente virtual nativo do Python (**venv**) em vez de **uv**.

B.1 É possível usar venv em vez de uv?

Sim. A documentação atual do Reflex recomenda **uv** como opção padrão, mas declara explicitamente que **venv**, Conda e Poetry são alternativas. Portanto, **venv** é uma abordagem suportada e não incompatível com o Reflex.

A diferença principal é o gerenciamento do ambiente e das dependências:

Abordagem	Ambiente	Instalação	Execução
Padrão da documentação	uv	uv add reflex	uv run reflex run
Este apêndice	venv	pip install reflex	reflex run

Para um projeto acadêmico, **venv** pode ser interessante por utilizar ferramentas nativas do Python e reduzir a quantidade de ferramentas novas.

Recomendação: mantenha o projeto consistente com **venv** + **pip**; não misture **uv** e **pip**.

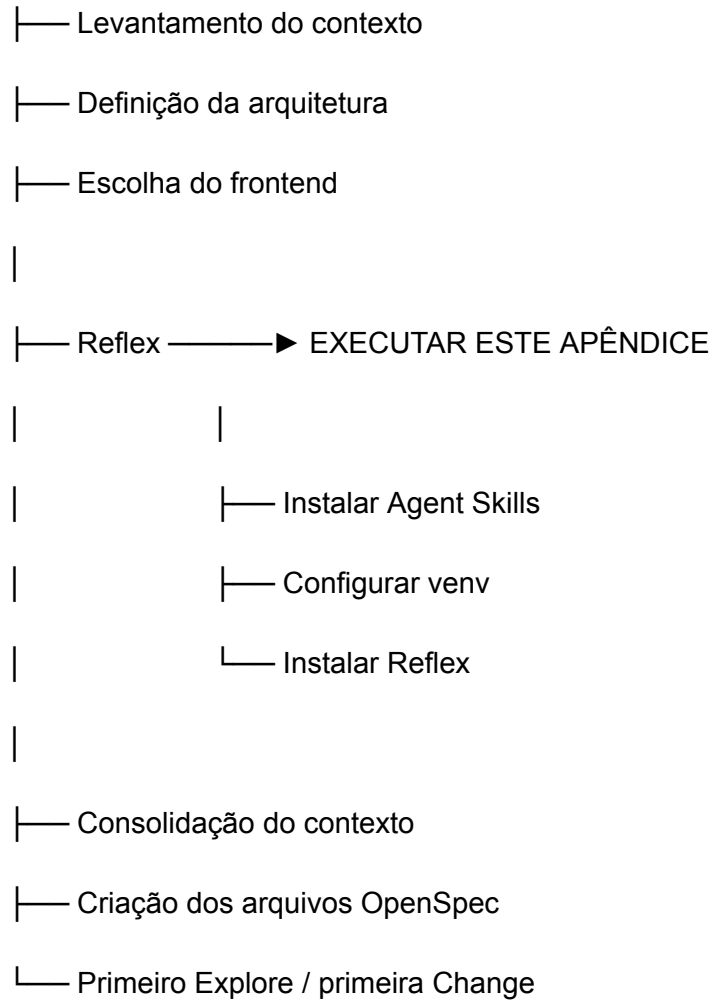
B.2 Onde este apêndice entra no fluxo

Execute depois da definição da arquitetura e da escolha de Reflex, e antes da consolidação final de **project-overview.md**, **config.yaml** e **AGENTS.md**. A parte de recuperação em outra máquina é um cenário posterior e independente da configuração inicial.

Manual principal

|

└─ Definição do projeto



Não execute o Apêndice A.

B.3 Pré-requisitos

Tenha instalado:

- Python 3.10 ou superior;
- VS Code;
- Git;
- acesso ao terminal;
- acesso à internet.

Verifique:

```
python --version
```

No Windows, se necessário:

```
py --version
```

B.4 Instalar as Agent Skills do Reflex

O Reflex mantém **Agent Skills** com orientações atualizadas para agentes de programação. Essas skills fornecem ao agente conhecimento e procedimentos específicos para trabalhar com Reflex, incluindo documentação da tecnologia, configuração do ambiente Python e gerenciamento do processo da aplicação.

Neste projeto, considera-se o uso do **GitHub Copilot no modo Agent integrado ao VS Code**.

B.4.1 Instalação

Na pasta raiz do projeto, execute:

```
npx skills add reflex-dev/agent-skills
```

A instalação deverá criar a estrutura de skills no projeto:

```
.agents/  
└── skills/  
    ├── reflex-docs/  
    ├── setup-python-env/  
    └── reflex-process-management/
```

As skills relevantes para este apêndice são:

- **reflex-docs** — documentação atualizada do Reflex;
- **setup-python-env** — configuração do ambiente Python;
- **reflex-process-management** — execução, compilação, recarregamento e depuração da aplicação Reflex.

B.4.2 Verificar as skills no GitHub Copilot

Depois da instalação, abra o projeto no VS Code e, no **GitHub Copilot Chat**, execute:

```
/skills
```

Confirme que aparecem as três skills:

reflex-docs

setup-python-env

reflex-process-management

Não prossiga com a configuração do Reflex caso essas skills não estejam disponíveis.

A verificação é importante porque a instalação dos arquivos no projeto e o reconhecimento das skills pelo agente são etapas distintas.

B.4.3 Atualização das skills

As skills são mantidas no repositório oficial do Reflex e podem receber atualizações.

Como elas estão instaladas no escopo deste projeto, uma atualização deve ser feita conscientemente quando houver necessidade, especialmente quando o projeto passar a utilizar uma versão mais recente do Reflex ou quando houver mudanças relevantes na documentação e nos procedimentos das skills.

Depois de atualizar as skills, verifique novamente no GitHub Copilot:

/skills

As alterações da pasta `.agents/skills/` devem ser tratadas como parte da configuração do projeto e podem ser versionadas no Git.

B.5 Criar o ambiente virtual

Na pasta do projeto:

```
python -m venv .venv
```

No Windows, se `python` não estiver disponível:

```
py -m venv .venv
```

B.6 Ativar o ambiente virtual

Windows PowerShell

```
.venv\Scripts\Activate.ps1
```

Windows CMD

```
.venv\Scripts\activate.bat
```

macOS/Linux

```
source .venv/bin/activate
```

Confirme que o ambiente está ativo. Normalmente o terminal passa a exibir (.venv).

B.7 Atualizar o pip

Com o ambiente ativo:

```
python -m pip install --upgrade pip
```

B.8 Instalar o Reflex

```
pip install reflex
```

Verifique:

```
reflex --version
```

Se o comando não for reconhecido, confirme se o .venv está ativo.

Também é possível verificar:

```
python -m pip show reflex
```

B.9 Inicializar o projeto Reflex

Execute:

```
reflex init
```

Quando forem apresentados os templates, escolha:

0 - A blank Reflex app

A documentação atual utiliza essa opção como ponto de partida para os tutoriais básicos.

Atenção: `reflex init` não é executado a cada sessão

`reflex init` é usado para inicialização/scaffolding. Depois da inicialização, use:

```
reflex run
```

A CLI oficial define `reflex init` para criar/reinicializar a aplicação e `reflex run` para executá-la.

B.10 Executar a aplicação

Com `.venv` ativo:

```
reflex run
```

A aplicação deverá ficar disponível, por padrão, em:

`http://localhost:3000`

O modo de desenvolvimento possui hot reload.

B.11 Verificar a estrutura

A inicialização cria uma estrutura própria do Reflex. Entre os elementos importantes podem estar:

`meu-projeto/`

|

|— `.venv/`

|— `assets/`

|— `rxconfig.py`

|

└─ `meu_projeto/`

└─ `meu_projeto.py`

A estrutura exata pode variar conforme a versão e o nome do projeto. `rxconfig.py` é utilizado para configuração da aplicação e o diretório do projeto contém o código da aplicação.

B.12 Registrar as dependências

Como esta versão utiliza `pip` + `venv`, não será utilizado `uv.lock`.

Depois de instalar as dependências necessárias:

```
pip freeze > requirements.txt
```

O `requirements.txt` deve ser versionado no Git.

Para recriar o ambiente em outra máquina:

```
python -m venv .venv
```

Ative o ambiente e execute:

```
pip install -r requirements.txt
```

B.13 Reutilizar o projeto em outra máquina (B.13 até B.18) --- Se não for o caso, pular para B.19

Esta seção descreve um cenário diferente da configuração inicial.

Suponha que o projeto já esteja versionado no Git e que você queira continuar o desenvolvimento em outro computador.

O ambiente `.venv` **não deve ser versionado** e, portanto, não será transferido pelo Git. Na nova máquina, será necessário criar um novo ambiente virtual e instalar nele as dependências registradas em `requirements.txt`.

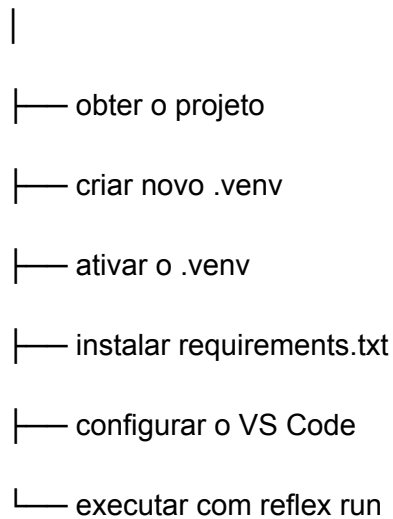
O fluxo é:

Projeto versionado no Git

|



outra máquina



B.13.1 Obter o projeto

Se o projeto estiver em um repositório remoto, clone-o na nova máquina:

```
git clone <URL_DO_REPOSITORIO>
```

Entre na pasta do projeto:

```
cd <NOME_DO_PROJETO>
```

Se o projeto já estiver disponível localmente por outro meio, apenas abra a pasta correspondente no terminal e no VS Code.

B.13.2 Não execute `reflex init`

Ao recuperar um projeto Reflex existente, **não execute `reflex init` novamente.**

O `reflex init` foi utilizado na criação inicial do projeto. Nesta situação, estamos apenas reconstruindo o ambiente de desenvolvimento para trabalhar com um projeto que já existe.

O objetivo agora é:

projeto existente

+

novo ambiente Python

+

dependências do projeto



ambiente pronto para continuar o desenvolvimento

B.14 Criar o novo ambiente virtual

Na nova máquina, crie um novo ambiente virtual na pasta do projeto:

```
python -m venv .venv
```

No Windows, se **python** não estiver disponível:

```
py -m venv .venv
```

O fato de o **.venv** ser recriado não significa que o projeto esteja sendo recriado. Estamos apenas criando o ambiente Python local necessário para executar o projeto.

B.14.1 Ativar o ambiente virtual

Windows PowerShell

```
.venv\Scripts\Activate.ps1
```

Windows CMD

```
.venv\Scripts\activate.bat
```

macOS/Linux

```
source .venv/bin/activate
```

Confirme que o ambiente está ativo. Normalmente o terminal passa a exibir (**.venv**).

B.15 Instalar as dependências do projeto

Com o novo **.venv** ativo, atualize o **pip**:

```
python -m pip install --upgrade pip
```

Depois instale as dependências registradas pelo projeto:

```
pip install -r requirements.txt
```

O `requirements.txt` representa as dependências que foram registradas na máquina em que o projeto foi configurado.

Não é necessário instalar o Reflex manualmente com `pip install reflex` se ele já estiver listado no `requirements.txt`. Nesse caso, ele será instalado junto com as demais dependências.

B.15.1 Verificar o Reflex

Depois da instalação, verifique:

```
reflex --version
```

Se o comando não for reconhecido, confirme se o `.venv` está ativo.

B.16 Configurar o projeto no VS Code

Abra a pasta do projeto no VS Code.

Selecione o interpretador Python do novo ambiente:

1. abra a Command Palette;
2. procure **Python: Select Interpreter**;
3. selecione o interpretador localizado em `.venv`.

Em uma nova sessão do terminal, lembre-se de ativar novamente o `.venv`.

B.17 Testar a aplicação recuperada

Com o ambiente ativo, execute:

```
reflex run
```

A aplicação deverá ficar disponível, por padrão, em:

```
http://localhost:3000
```

Se a aplicação abrir corretamente, o ambiente da nova máquina está funcional.

Neste cenário, o comando utilizado para continuar o trabalho é `reflex run`, e não `reflex init`.

B.18 Voltar ao fluxo principal

Depois de concluir a configuração inicial ou a recuperação do projeto em outra máquina, retorne ao fluxo principal do manual.

O objetivo é que o ambiente esteja pronto para a próxima etapa do desenvolvimento, sem misturar a configuração do ambiente com a implementação das funcionalidades.

B.19 Conceitos iniciais

Antes de implementar funcionalidades, conheça minimamente:

- componentes;
- `State`;
- variáveis de estado;
- event handlers;
- páginas;
- rotas;
- configuração da aplicação;
- diferença entre código executado no backend e código compilado para o frontend.

O Reflex apresenta componentes, State, eventos e páginas como conceitos fundamentais.

B.20 Registrar Reflex no `project-overview.md`

Na arquitetura tecnológica, registre:

Frontend: Reflex

E:

Reflex será utilizado como tecnologia exclusiva para

implementação do frontend da aplicação.

Não transforme o `project-overview.md` em manual do Reflex.

B.21 Registrar Reflex no `config.yaml`

Inclua no contexto:

Frontend:

Reflex é a tecnologia obrigatória para implementação das interfaces frontend do projeto.

Não introduzir outra tecnologia de frontend sem uma mudança explícita da decisão arquitetural.

B.22 Registrar a restrição no **AGENTS.md**

Inclua uma regra equivalente a:

Frontend

O frontend do projeto deve ser implementado exclusivamente com Reflex.

Utilize os mecanismos próprios do Reflex para componentes, estado, eventos, páginas e interação.

Não introduza outra tecnologia de frontend para substituir ou complementar o Reflex, salvo quando houver uma alteração arquitetural explicitamente aprovada.

B.23 Atenção ao **AGENTS.md**

O Reflex possui documentação e recursos próprios para agentes de IA. Como o projeto possui um **AGENTS.md** definido pelo manual principal:

- não substitua automaticamente o arquivo do projeto;
- examine qualquer conteúdo gerado pelo Reflex;
- preserve as regras gerais do projeto;
- incorpore apenas instruções compatíveis com elas.

As instruções específicas do Reflex são complementares ao OpenSpec.

B.24 Skills e documentação do projeto

As Agent Skills não substituem os artefatos que definem o contexto e as regras do projeto.

Elas são complementares a:

project-overview.md

domain-model.md

AGENTS.md

config.yaml

openspec/specs/

openspec/changes/

Em particular:

- `project-overview.md` define o contexto e a visão geral do projeto;
- `domain-model.md` registra o modelo de domínio;
- `config.yaml` registra decisões e contexto de configuração;
- `AGENTS.md` contém as regras gerais para o agente;
- `openspec/specs/` contém as especificações do sistema;
- `openspec/changes/` contém as mudanças em desenvolvimento;
- as Agent Skills fornecem conhecimento e procedimentos especializados para o agente trabalhar com Reflex.

As skills, portanto, **não substituem o OpenSpec nem as regras do projeto.**

B.25 Git

Versione:

requirements.txt

rxconfig.py

código-fonte

AGENTS.md

docs/

openspec/

Não versione:

.venv/

B.25.1 Configurar o `.gitignore`

Para evitar que a pasta `.venv/` seja adicionada acidentalmente ao repositório, ela deve ser incluída no arquivo `.gitignore` do projeto.

Na raiz do projeto, crie ou edite o arquivo chamado:

`.gitignore`

Adicione a seguinte linha:

`.venv/`

O conteúdo mínimo do arquivo será:

`.venv/`

A partir desse momento, o Git ignorará a pasta `.venv/` e seus arquivos.

Importante: o `.gitignore` também deve ser versionado. Ele faz parte da configuração do projeto e garante que outros desenvolvedores utilizem as mesmas regras de exclusão.

Assim, o projeto terá:

sistema/

|— `.gitignore` ← versionar

|— `.venv/` ← NÃO versionar

|— `requirements.txt` ← versionar

|— `rxconfig.py` ← versionar

└─ ...

Se a pasta `.venv/` já tiver sido adicionada ao Git antes da criação do `.gitignore`, será necessário removê-la do controle de versão com:

```
git rm -r --cached .venv
```

Esse comando **não apaga o ambiente virtual do computador**; ele apenas remove a pasta do controle de versão do Git.

B.26 Cuidados com `pip freeze`

`pip freeze` registra as versões instaladas no ambiente. Portanto, evite instalar pacotes aleatórios.

Quando uma funcionalidade exigir uma nova biblioteca:

1. verifique se ela é necessária;
2. instale-a no `.venv`;
3. teste;
4. atualize `requirements.txt`;
5. versiona a alteração junto com a mudança correspondente.

B.27 `venv` versus `uv`

A escolha de `venv` **não altera os conceitos fundamentais do Reflex nem a arquitetura do sistema**.

A principal diferença está no gerenciamento de dependências.

Com `venv`:

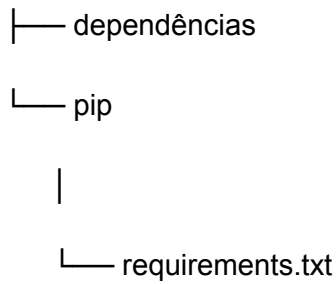
Python

|

└─ .venv

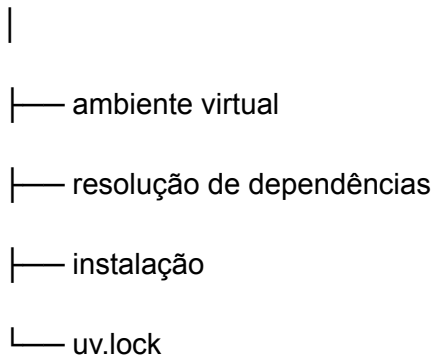
|

└─ Reflex



Com **uv**:

uv



A documentação atual do Reflex recomenda **uv** como padrão, mas reconhece **venv** como alternativa.

Para um projeto acadêmico, utilizar **venv** é perfeitamente razoável se todos os integrantes seguirem o mesmo procedimento.

A principal desvantagem é não utilizar os recursos de gerenciamento e lockfile oferecidos pelo **uv**.

B.28 O que não fazer

Neste momento, não é necessário:

- executar **reflex init** novamente a cada sessão;
- criar todas as páginas;
- criar todo o State;
- implementar todas as funcionalidades;
- definir toda a arquitetura do frontend;
- introduzir React, Vue, Angular ou outro framework;
- utilizar **uv**;
- criar **uv.lock** sem adotar **uv** como gerenciador do projeto.

B.29 Retorno ao manual principal

Depois de concluir este apêndice, retorne ao ponto indicado no manual principal.

Resultado esperado:

Ambiente Reflex funcional

+

venv configurado

+

requirements.txt criado

+

Escolha tecnológica registrada

+

Contexto OpenSpec preparado

↓

Continuação do manual principal

↓

Primeiro Explore / primeira Change

B.30 Referências oficiais

- Reflex — Installation: <https://reflex.dev/docs/getting-started/installation>
- Reflex — Project Structure: <https://reflex.dev/docs/getting-started/project-structure>
- Reflex — Basics: <https://reflex.dev/docs/getting-started/basics>
- Reflex — CLI: <https://reflex.dev/docs/api-reference/cli>
- Reflex — Configuration: <https://reflex.dev/docs/advanced-onboarding/configuration>
- Reflex — Agent Skills: <https://reflex.dev/docs/ai/integrations/skills/>